

- 🏗️ Global Multicloud Healthcare & Financial Transaction Clearing Engine
 - 📄 Table of Contents
 - 🏛️ Architecture Overview
 - 🌐 High-Level Topology
 - 🔧 Component Matrix
 - 🔒 Network Architecture
 - Cross-Cloud VPN Mesh
 - Subnet Architecture
 - 📊 Data Flow Architecture
 - 🛡️ Compliance Architecture
 - HIPAA Controls
 - SOX Controls
 - GDPR Controls
 - 👤 Identity Architecture (Azure AD)
 - 📁 Directory Structure
 - ✅ Prerequisites
 - 🚀 Deployment Guide
 - Step 1: Configure Variables
 - Step 2: Initialize Terraform
 - Step 3: Plan & Review
 - Step 4: Apply
 - Step 5: Configure ArgoCD
 - 🔄 GitOps & ArgoCD
 - 🔥 Disaster Recovery

🏗️ Global Multicloud Healthcare & Financial Transaction Clearing Engine

Production-grade multicloud infrastructure for ingesting, validating, and clearing sensitive medical billing transactions across AWS, Azure, and GCP.

🔧 Terraform **>=1.7.0** 🌩️ AWS **Primary Active** ☁️ Azure **Hot Standby** 🌐 GCP **Compliance**

HIPAA | SOX | GDPR | PCI-DSS **Compliant**

📄 Table of Contents

- [Architecture Overview](#)
- [High-Level Topology](#)
- [Component Matrix](#)
- [Network Architecture](#)
- [Data Flow Architecture](#)
- [Compliance Architecture](#)
- [Identity Architecture \(Azure AD\)](#)
- [Directory Structure](#)
- [Prerequisites](#)
- [Deployment Guide](#)
- [GitOps & ArgoCD](#)
- [Disaster Recovery](#)



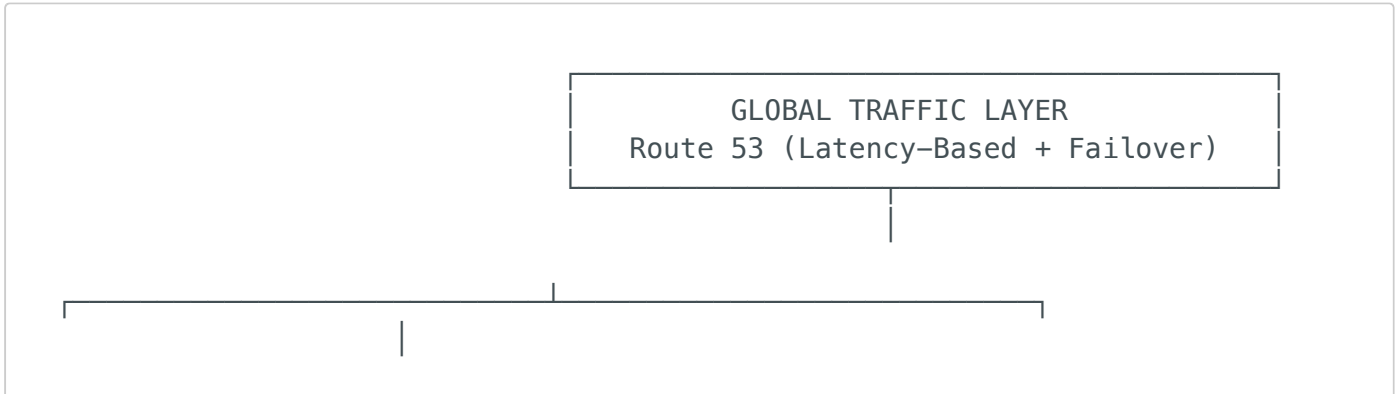
Architecture Overview

This system implements a **three-cloud active/standby/compliance** architecture:

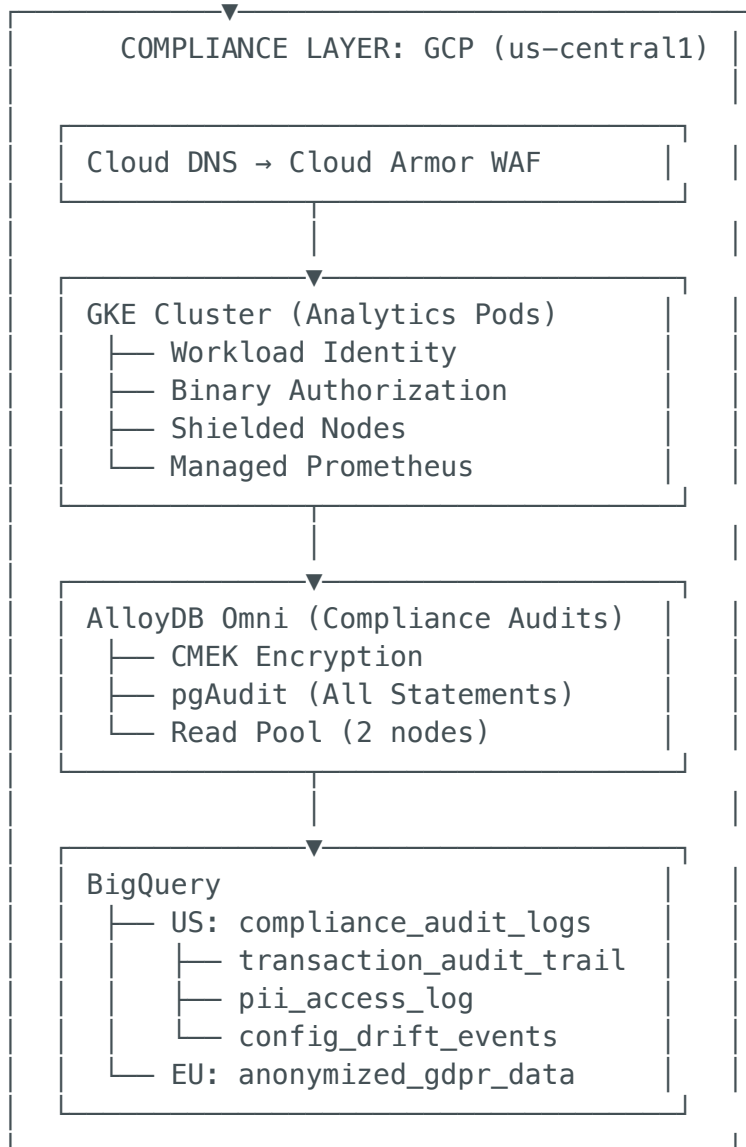
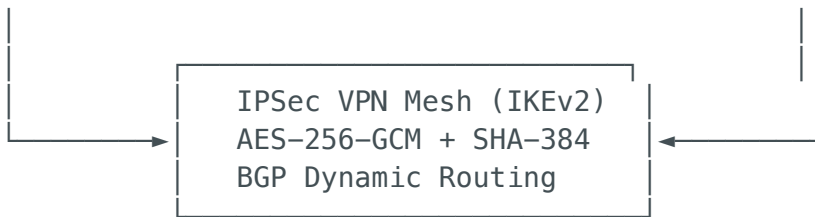
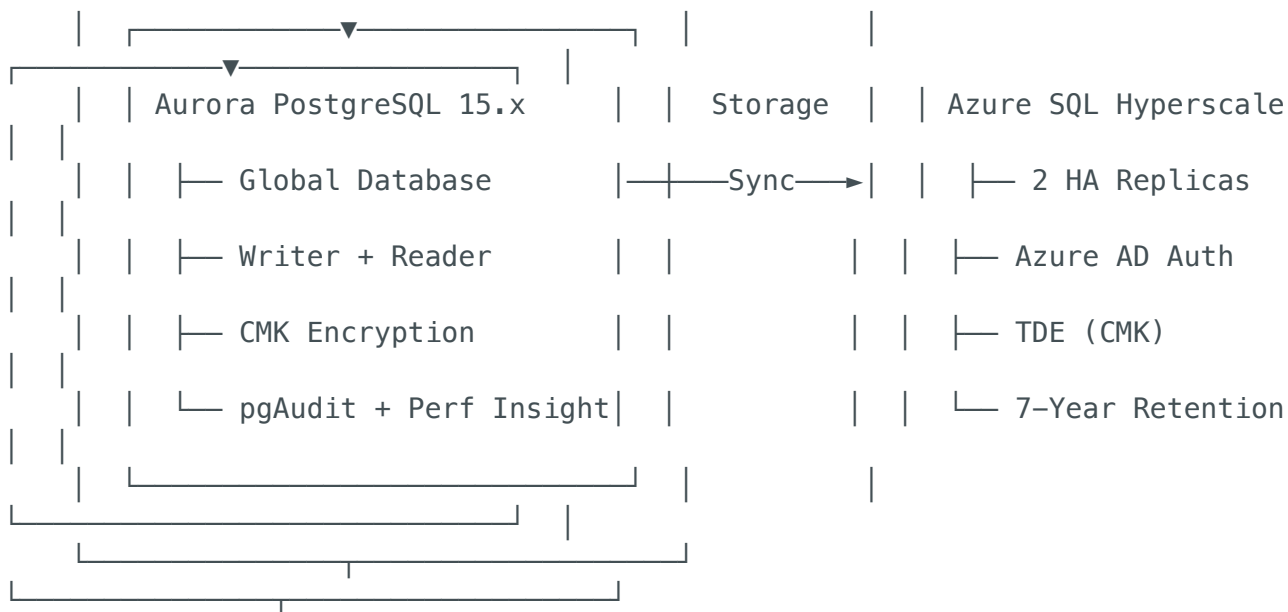
Cloud	Role	Services
AWS	Primary Active Site	EKS, Aurora PostgreSQL, ElastiCache Redis, CloudFront + WAF, Route 53
Azure	Hot Standby Site	AKS, Azure SQL Hyperscale, Azure Cache for Redis, Front Door + WAF
GCP	Compliance & Analytics	GKE, AlloyDB, BigQuery, Cloud Armor, Cloud KMS



High-Level Topology









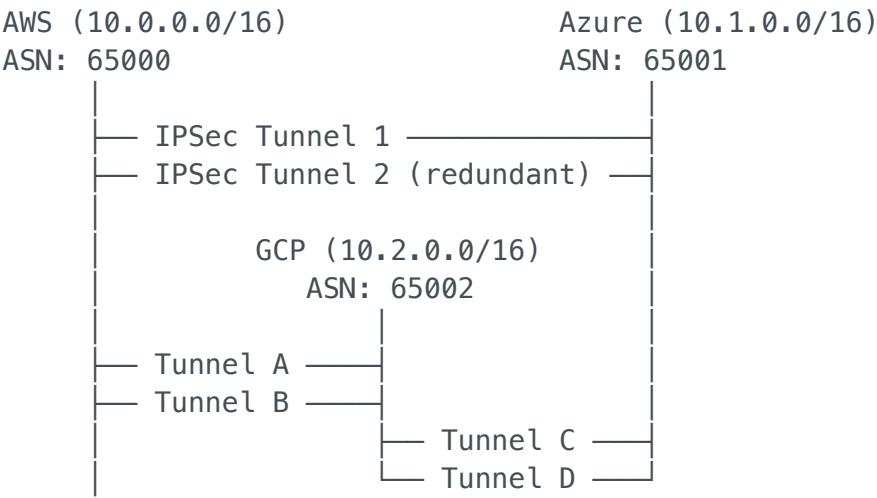
Component Matrix

Component	AWS (Primary)	Azure (Hot Standby)	GCP (Compliance)
Global DNS	Route 53	Azure DNS	Cloud DNS
CDN & Edge	CloudFront + WAF	Front Door Premium + WAF	Cloud CDN + Armor
Compute	EKS + Managed Nodes	AKS + System/User Pools	GKE Standard
Service Mesh	Istio (STRICT mTLS)	Istio (STRICT mTLS)	Istio (STRICT mTLS)
GitOps	ArgoCD	ArgoCD	ArgoCD
Database	Aurora PostgreSQL 15	SQL Hyperscale	AlloyDB PostgreSQL
Caching	ElastiCache Redis 7.x	Cache for Redis Premium	—
Analytics	—	—	BigQuery
Secrets	KMS + IRSA	Key Vault + Workload ID	Cloud KMS + WI
VPN	VPN Gateway (BGP 65000)	VPN Gateway (BGP 65001)	HA VPN (BGP 65002)
Compliance	CloudTrail + Config	Azure Policy + Defender	Audit Logs + SCC
Identity	IAM + OIDC	Azure AD (Central)	IAM + WI Pools



Network Architecture

Cross-Cloud VPN Mesh

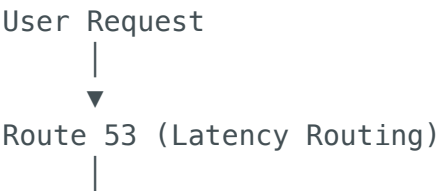


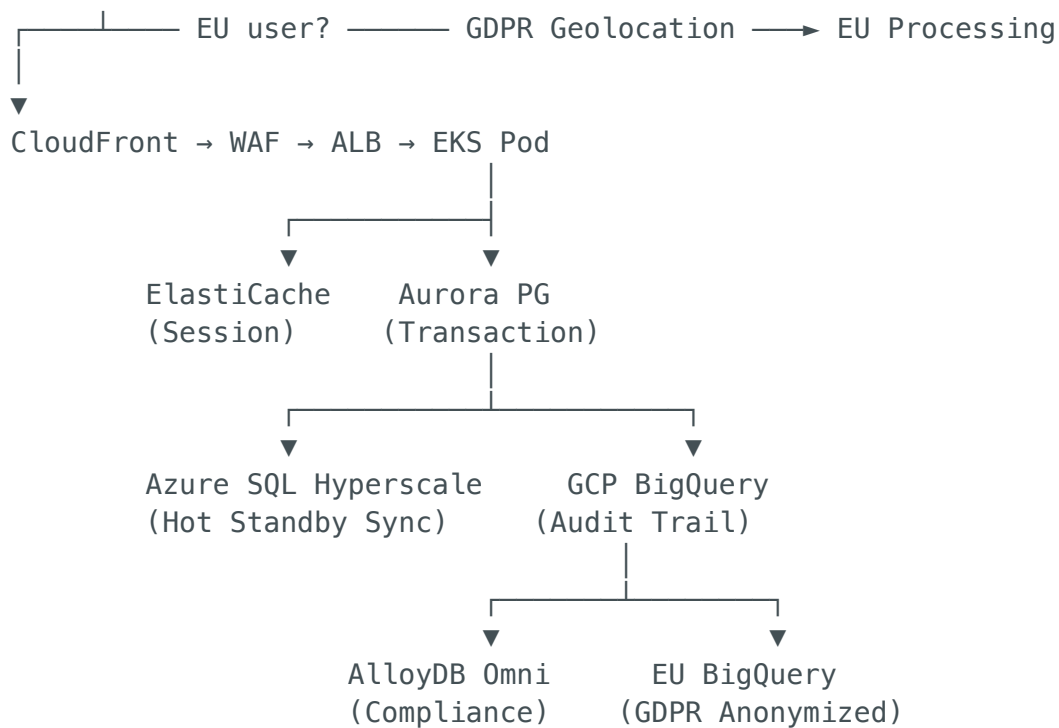
All tunnels: IKEv2, AES-256-GCM, SHA-384, DH Group 20

Subnet Architecture

Cloud	Subnet	CIDR	Purpose
AWS	Public (x2)	10.0.0.0/24, 10.0.1.0/24	ALB, NAT Gateways
AWS	Private-App (x2)	10.0.10.0/24, 10.0.11.0/24	EKS Worker Nodes
AWS	Private-Data (x2)	10.0.20.0/24, 10.0.21.0/24	Aurora, ElastiCache
Azure	AKS System	10.1.0.0/24	AKS System Pool
Azure	AKS User	10.1.1.0/24	AKS User Pool
Azure	Data	10.1.10.0/24	SQL, Redis, Key Vault
Azure	Gateway	10.1.255.0/24	VPN Gateway
GCP	GKE Nodes	10.2.0.0/20	GKE Node Pool
GCP	Data	10.2.16.0/20	AlloyDB, Private Services

Data Flow Architecture





Compliance Architecture

HIPAA Controls

Control	Implementation
Data at Rest Encryption	AWS KMS (CMK), Azure Key Vault (CMK), GCP Cloud KMS (CMEK)
Data in Transit	Istio mTLS (STRICT mode) + IPsec VPN (AES-256-GCM)
Access Controls	IAM Roles, Azure RBAC, GCP IAM + Workload Identity
Audit Logging	CloudTrail, Azure Monitor, GCP Audit Logs → BigQuery
Network Isolation	VPC/VNet with NACLs/NSGs, private subnets, no public DB access
Backup & Recovery	Aurora: 35-day, Azure SQL: 7-year, AlloyDB: 35-day + PITR

SOX Controls

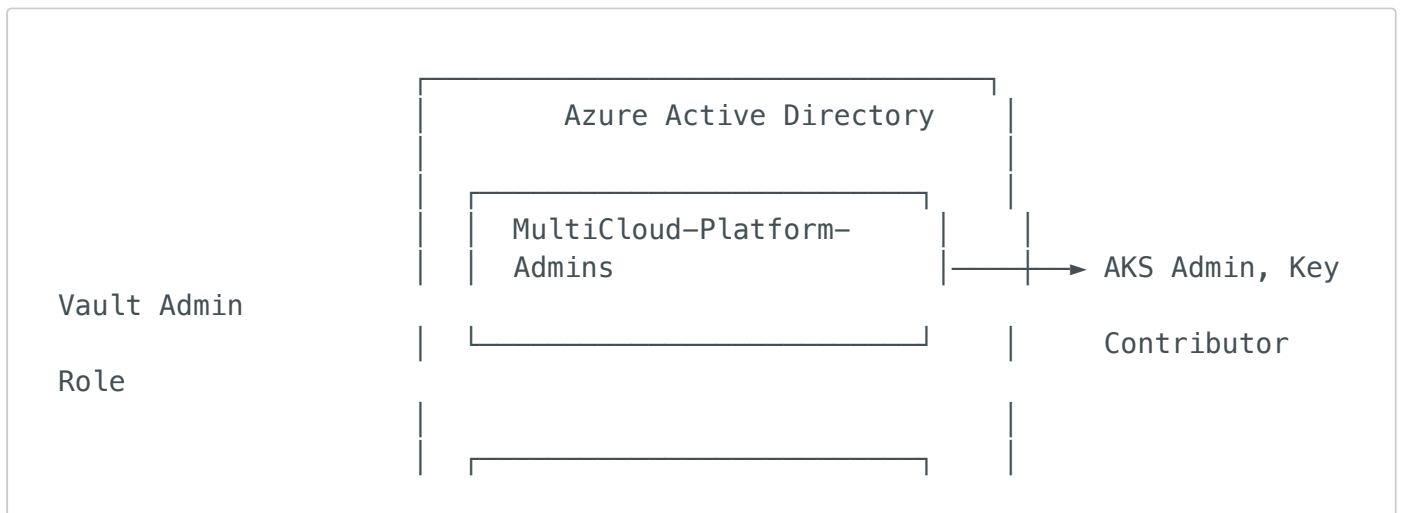
Control	Implementation
Immutable Audit Trail	CloudTrail (S3 + Glacier, 7yr), Azure Activity Log, GCP → BigQuery
Separation of Duties	Admin/Dev/Compliance Azure AD groups, IAM role boundaries
Change Detection	AWS Config Rules, Azure Policy, GCP Org Policies + Drift Alerts
Financial Auditability	pgAudit on all DB writes, Performance Insights, BigQuery analytics

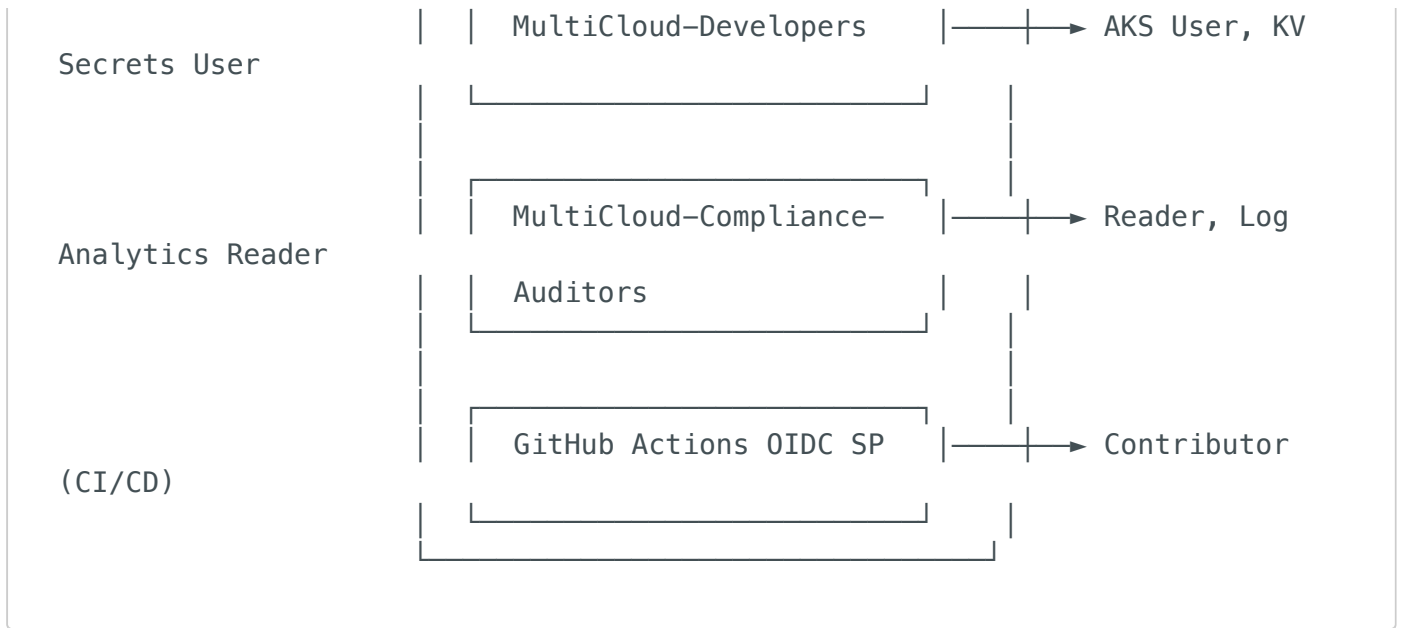
GDPR Controls

Control	Implementation
Data Sovereignty	Route 53 geolocation routing, BigQuery EU dataset, KMS EU keyring
Right to Erasure	Tokenized PII, anonymization in microservices before cross-cloud sync
Egress Control	Istio Sidecar REGISTRY_ONLY mode + whitelisted ServiceEntries
Data Minimization	Pod-level egress blocks unauthorized API calls

Identity Architecture (Azure AD)

Azure Active Directory serves as the **centralized identity provider** across all three clouds:





Directory Structure

```

MulitCloudProject/
├── .github/workflows/
│   ├── aws-deploy.yml           # Full multicloud DevSecOps pipeline
│   ├── azure-deploy.yml        # Azure DevSecOps pipeline (Security → Plan
→ Apply → Compliance)
│   └── gcp-deploy.yml          # GCP DevSecOps pipeline (Security → Plan →
Apply → Compliance)
├── terraform/
│   ├── bootstrap/              # State backend provisioning (run once)
│   │   ├── providers.tf        # AWS, Azure, GCP provider config
│   │   ├── main.tf             # S3+DynamoDB, Azure Storage, GCS bucket
│   │   ├── variables.tf        # Bootstrap configuration
│   │   └── outputs.tf          # Backend config snippets
│   └── modules/
│       ├── aws_infra/          # VPC, EKS, Aurora, ElastiCache, CloudFront
│       │   ├── main.tf
│       │   ├── variables.tf
│       │   ├── outputs.tf
│       │   ├── vpc.tf
│       │   ├── eks.tf
│       │   ├── aurora.tf
│       │   ├── elasticache.tf
│       │   ├── cloudfront.tf
│       │   ├── route53.tf
│       │   ├── kms.tf
│       │   ├── vpn.tf
│       │   ├── compliance.tf
│       │   └── iam.tf
│       └── azure_infra/        # VNet, AKS, SQL, Redis, Front Door, Azure
AD
│   ├── main.tf
│   └── variables.tf
  
```

```

├── outputs.tf
├── vnet.tf
├── aks.tf
├── sql.tf
├── redis.tf
├── frontdoor.tf
├── keyvault.tf
├── vpn.tf
├── compliance.tf
├── azure_ad.tf
├── gcp_infra/                # VPC, GKE, AlloyDB, BigQuery
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── vpc.tf
│   ├── gke.tf
│   ├── alloydb.tf
│   ├── bigquery.tf
│   ├── kms.tf
│   ├── vpn.tf
│   └── compliance.tf
├── environments/
│   └── production/
│       ├── main.tf
│       ├── providers.tf
│       ├── variables.tf
│       ├── outputs.tf
│       └── terraform.tfvars
├── gitops/
│   └── apps/
│       ├── base/            # Shared Istio + namespace configs
│       ├── aws/             # ArgoCD + Istio egress for EKS
│       ├── azure/           # ArgoCD + Istio egress for AKS
│       └── gcp/              # ArgoCD + Istio egress for GKE
├── docs/
│   └── RUNBOOK.md           # Comprehensive Operations Runbook
└── README.md                # This file

```



Prerequisites

1. **Cloud Accounts:** Active AWS, Azure, and GCP accounts with billing enabled
2. **Terraform:** $\geq 1.7.0$ installed locally
3. **OIDC Federation** (for CI/CD):
 - AWS: IAM OIDC Identity Provider + IAM Role for GitHub Actions
 - Azure: App Registration + Federated Credentials
 - GCP: Workload Identity Pool + Provider
4. **DNS Domain:** A registered domain for Route 53 hosted zone

5. **S3 Backend:** Create `enterprise-global-tfstate` S3 bucket + `terraform-lock` DynamoDB table
 6. **GitHub Secrets:** Configure all required secrets (see CI/CD pipeline)
-



Deployment Guide

Step 1: Configure Variables

```
cd terraform/environments/production
# Edit terraform.tfvars with your actual values
vim terraform.tfvars
```

Step 2: Initialize Terraform

```
terraform init
```

Step 3: Plan & Review

```
terraform plan -out=tfplan
```

Step 4: Apply

```
terraform apply tfplan
```

Step 5: Configure ArgoCD

After clusters are provisioned, install ArgoCD and apply the GitOps manifests:

```
# On each cluster (EKS, AKS, GKE):
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml

# Apply application manifests
kubectl apply -f gitops/apps/base/
kubectl apply -f gitops/apps/<cloud>/
```



GitOps & ArgoCD

ArgoCD manages the Kubernetes state after Terraform provisions infrastructure:

- **Auto-sync:** Changes pushed to `gitops/apps/` are automatically applied
- **Self-heal:** Manual cluster changes are reverted to Git state
- **Prune:** Orphaned resources are automatically cleaned up



Disaster Recovery

Scenario	RTO	RPO	Procedure
AWS Region Failure	5 min	0 (sync replication)	Route 53 auto-failover to Azure
Single AZ Failure	0	0	Multi-AZ architecture handles automatically
Database Corruption	30 min	5 min (PITR)	Point-in-time recovery from Aurora
Full Cloud Outage	15 min	~1 min	Manual DNS cutover to Azure Front Door

See [docs/RUNBOOK.md](#) for detailed operational procedures.